

ReasoningEconomicsEnv: Learning to Allocate Thinking Tokens Under a Shared Budget

Yashaswi Sharma, Harshawn Singh, Khushi Kumari, Ryuichi Lun, Prabhu Pugalenthi

University of Southern California, CA, USA

{yashaswi, harshawn, kkumari, ryuichi.y.lun, pugalent}@usc.edu

Abstract

Scaling test-time compute has emerged as a key driver of performance in large language models (LLMs), yet existing approaches largely treat compute allocation as fixed or externally controlled, failing to capture the sequential and budget-constrained nature of real-world inference. We introduce **ReasoningEconomicsEnv**, a reinforcement learning environment for training LLMs to *jointly allocate and utilize reasoning tokens* under a shared budget across multiple tasks. In our formulation, a single model acts as both allocator and solver, learning to dynamically trade off reasoning depth against correctness over multi-question episodes. The environment supports both hard budget constraints and soft penalty-based regimes, enabling fine-grained control over efficiency–accuracy tradeoffs. We further develop **ReasoningEconomicsPT**, a scalable post-training pipeline implementing Group Relative Policy Optimization (GRPO) for long-horizon, budget-aware reasoning. Our system integrates diverse mathematical reasoning datasets and supports distributed training across heterogeneous GPU clusters. Together, these contributions provide a practical framework for studying *adaptive inference policies* in LLMs and lay the groundwork for learning cost-aware reasoning strategies under realistic deployment constraints.

1 Introduction

Recent advances in large language models (LLMs) have shown that increasing test-time compute—particularly through chain-of-thought reasoning—can substantially improve performance on complex tasks. However, these gains come with increased latency and computational cost, making unconstrained reasoning impractical in real-world deployments. This has motivated growing interest in *adaptive inference*, where models dynamically adjust their reasoning effort to balance accuracy and efficiency.

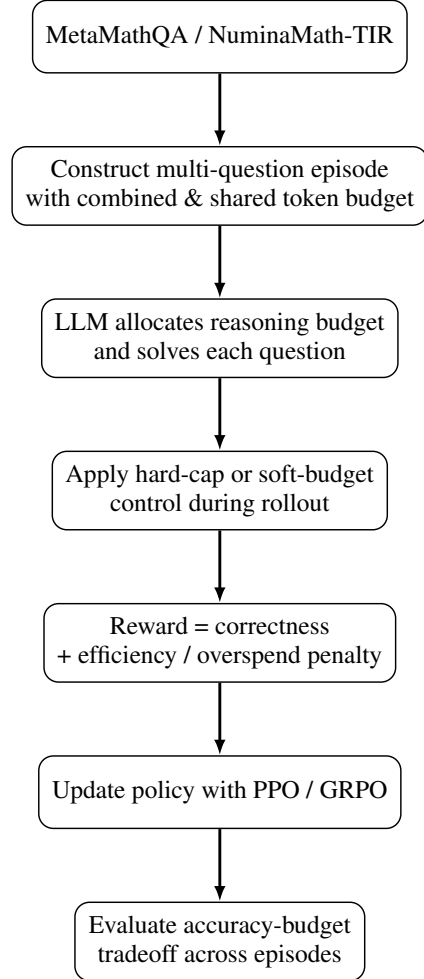


Figure 1: Flow of the ReasoningEconomicsEnv framework. The model allocates reasoning tokens per query under a shared budget and is trained to optimize both correctness and efficiency.

Despite this progress, current LLMs exhibit a fundamental limitation: they fail to calibrate reasoning effort to problem difficulty. In practice, this leads to two characteristic failure modes: *overthinking*, where models generate unnecessarily long reasoning traces on simple tasks, and *underthinking*, where models prematurely truncate reasoning on difficult problems, resulting in incorrect answers.

Existing approaches attempt to improve efficiency through prompt engineering, length-regularized reinforcement learning, or early-exit strategies, but these methods operate at the level of *individual queries*. They do not address a more realistic setting in which a model must allocate a finite compute budget across a *sequence* of tasks, where decisions on earlier problems directly constrain the resources available for later ones.

To address this gap, we introduce **ReasoningEconomicsEnv**, a reinforcement learning environment that trains LLMs to jointly perform *compute allocation and problem solving* under a shared budget constraint. In our formulation, a single model acts as both the allocator and solver within a multi-step episode consisting of multiple questions. At each step, the model decides how much reasoning to allocate before generating an answer, trading off local accuracy against global budget consumption. We implement both hard budget constraints and soft penalty-based regimes, and optimize policies using Group Relative Policy Optimization (GRPO) to encourage efficient long-horizon reasoning strategies. Figure 1 illustrates the overall framework.

Empirically, we instantiate our framework on mixed mathematical reasoning datasets (MetaMathQA and NuminaMath-TIR) and develop a scalable post-training pipeline for long-horizon reinforcement learning. Our system enables evaluation of accuracy–efficiency tradeoffs at the episode level and supports distributed training across heterogeneous GPU infrastructure. Initial results indicate that models trained in this environment learn non-trivial allocation strategies that balance correctness and compute usage, improving cumulative reward relative to baseline policies. These findings suggest that explicitly modeling compute as a shared resource is a promising direction for learning cost-aware reasoning in LLMs.

2 Related Work

Scaling Test-Time Compute for Reasoning A growing body of work shows that increasing test-time compute, particularly via chain-of-thought (CoT) reasoning, leads to substantial improvements in LLM performance on complex tasks (Snell et al., 2024; Tong et al., 2024). These approaches primarily treat compute as a monotonic resource—more tokens generally yield better results—and focus on scaling inference rather than optimizing its al-

location. In contrast, our work treats compute as a *constrained resource* and studies how a model can learn to allocate it adaptively across multiple decisions within a single episode.

Prompt-Based and Budget-Constrained Reasoning Prompting-based approaches such as token-budget prompting (Han et al., 2025), Chain-of-Draft (Xu et al., 2025), and constrained CoT methods (Renze and Guven, 2024) attempt to limit reasoning length by imposing explicit instructions or heuristics on generation. While effective in reducing verbosity, these methods rely on fixed or externally specified budgets and do not provide a mechanism for dynamic adaptation based on problem difficulty. Our approach differs in that the model itself learns a policy for allocating reasoning tokens, rather than following a predetermined budget or heuristic constraint.

RL for Efficient Reasoning Recent work applies reinforcement learning to encourage shorter or more efficient reasoning traces by incorporating length-based rewards or penalties (Shao et al., 2024). Methods such as O1-Pruner (Luo et al., 2025), DAST (Shen et al., 2025), and SelfBudgeter (Li et al., 2025) optimize the tradeoff between accuracy and token usage by shaping the reward function at the level of individual queries. However, these approaches treat each problem independently and do not model interactions between decisions across multiple tasks. In contrast, our formulation introduces a *shared episode-level budget*, where allocating more compute to one problem directly reduces the resources available for subsequent problems, requiring the model to learn long-horizon allocation strategies.

Adaptive Inference and Early-Exit Methods

Adaptive inference methods, including early-exit strategies and dynamic stopping rules like Dynasor-CoT (Fu et al., 2024) and Budget Forcing (Muenighoff et al., 2025), aim to terminate reasoning once sufficient confidence is reached. These approaches improve efficiency by reducing unnecessary computation within a single reasoning trajectory. However, they operate locally and do not consider global resource constraints across multiple tasks. Our work extends this idea by embedding inference decisions within a sequential decision-making framework, where stopping behavior must be coordinated with future budget requirements.

Distillation and Compressed Reasoning Distillation based approaches, such as CoT-Valve (Ma et al., 2025), TokenSkip (Xia et al., 2025), and related methods, train models to produce shorter reasoning traces by learning from compressed or selectively pruned demonstrations. While these techniques reduce average token usage, they implicitly encode fixed reasoning patterns and do not allow the model to dynamically adjust reasoning effort at test time. In contrast, our method enables adaptive behavior by training a policy that conditions compute allocation on both the current problem and the remaining budget within an episode.

3 Problem Description

We study the problem of learning *cost-aware reasoning* in large language models under a shared compute budget. In standard long-reasoning settings, a model is evaluated on one question at a time, and its reasoning length is optimized locally for that query. However, this formulation does not capture realistic deployment scenarios in which a system must solve a sequence of tasks while operating under a finite token budget. In such settings, spending too many tokens on an easy question may leave insufficient budget for later difficult questions, whereas spending too few tokens early may reduce the chance of solving important problems correctly. Our goal is to train a model that can make these budget allocation decisions autonomously.

Formally, an episode consists of a sequence of math problems drawn from a fixed dataset distribution. At each step, the model observes the current question together with the remaining episode budget and produces both a reasoning trace and a final answer. The environment then returns a reward that depends on answer correctness and token usage. The key challenge is that the model must learn a policy that balances immediate accuracy against long-horizon budget conservation. This makes the task a sequential decision problem rather than a single-query generation problem. The core difficulty is not simply generating shorter answers, but learning *when to think longer* and *when to stop*. On easy questions, the model should avoid unnecessary computation; on difficult questions, it should spend enough tokens to maintain accuracy. We therefore frame reasoning as a resource allocation problem in which the model itself must act as both the solver and the allocator of its own test-time compute.

4 Methods

We implement **ReasoningEconomicsEnv**, a reinforcement learning environment designed to train a single language model to allocate reasoning tokens across multiple questions under a shared episode budget. The environment is built around multi-question episodes sampled from MetaMathQA (Yu et al., 2023) and NuminaMath-TIR (Numina Project Team, 2024). Each episode contains up to ten sequential math problems, and the model must decide how much reasoning to spend on each step before producing a final answer. The central design choice is that the model is responsible for both solving the current problem and implicitly managing future budget availability.

Episode construction. For each training episode, we sample a sequence of questions from the selected datasets and combine them into one shared-budget rollout. The episode begins with an initial token budget B_0 . After each question, the environment decrements the remaining budget according to the number of tokens used during reasoning and answer generation. The next question is then presented together with the updated budget state. This setup makes the allocation decision sequential: the policy must reason not only about the current question, but also about how much compute should be preserved for later questions in the same episode.

Action space and policy behavior. At each step, the policy generates a reasoning trace followed by a final answer. In our setting, the length of the reasoning trace is not fixed in advance; instead, it is governed by the model’s own generation behavior and the current budget constraint. The same model thus plays two roles: it acts as a solver for the current question and as an allocator that implicitly decides how much of the remaining budget to spend. This is the key difference from standard per-query efficient reasoning methods, which optimize each problem independently.

Reward formulation. We implement a dense reward signal combining per-step feedback and an episode-level bonus to guide the model’s allocation policy. At each step t , the model receives a per-step reward R_{step} defined as:

$$R_{step} = c_t + \gamma \left(1 - \frac{\tau_t}{\bar{\tau}}\right) I_t - \beta \max\left(0, \frac{\tau_t}{\bar{\tau}} - 1\right) - P_{over} \quad (1)$$

where τ_t is the number of tokens used, $\bar{\tau} = B_{total}/N$ is the “fair share” budget per question,

$c_t \in \{1.0, -0.1\}$ is the base correctness score, and $I_t \in \{0, 1\}$ indicates if the answer is correct. The hyperparameters γ and β scale the efficiency bonus and cost penalty, respectively. In **soft-budget mode**, an additional penalty $P_{over} = \lambda_{soft}(\tau_{over}/\bar{\tau})$ is applied if the budget is exceeded; in **hard-cap mode**, $P_{over} = 0$ but the episode may terminate early.

Upon completing the episode, the agent receives an episode-level bonus $R_{episode}$ to encourage global budget optimization:

$$R_{episode} = \lambda_{ep} \left(\frac{N_{correct}}{N} \right) \max \left(0, 1 - \left| \frac{\tau_{total}}{B_{total}} - U_{target} \right| \right) \quad (2)$$

where $N_{correct}$ is the total number of correctly answered questions, τ_{total} is the total tokens spent across the episode, U_{target} is the target budget utilization rate, and λ_{ep} controls the overall magnitude of the bonus.

Training objective. We train the policy using Group Relative Policy Optimization (GRPO). Given a batch of episode rollouts, the training loop compares responses relative to other samples in the same group and updates the policy toward trajectories that achieve higher reward. This is especially useful in our setting because the reward is not purely local: the value of a given step depends on how much budget remains for later questions. GRPO therefore provides a natural way to optimize the long-horizon allocation behavior required by the environment.

Implementation details. We developed the environment in a form that can be served over HTTP and used by a separate post-training pipeline. The training codebase, ReasoningEconomicsPT, supports distributed GRPO training across both Lambda Labs GPU instances and the USC CARC Discovery cluster. To make the system reproducible and robust, we added preflight checks, end-point validation, and workflow scripts for launching training jobs in both local and cluster settings.

Dataset preprocessing. We use MetaMathQA (Yu et al., 2023) and NuminaMath-TIR (Numina Project Team, 2024) as the core reasoning datasets because both contain mathematical problems that require structured intermediate reasoning. Before training, we normalize the examples into a consistent question-and-answer format, remove malformed or unusable entries, and construct mixed episodes from the cleaned pools. We also enforce a maximum episode length of ten questions and standard-

ize the prompt format so that each step contains only the current problem and the remaining budget state. This preprocessing ensures that the model sees a stable input structure while still being exposed to a diverse set of reasoning tasks.

Training and evaluation setup. During training, the environment tracks token consumption, answer correctness, and remaining budget at each step. We use these signals to compute the episode reward and update the policy. For evaluation, we measure how well a trained model trades off accuracy and token use across complete episodes, rather than on isolated single-question benchmarks. This makes it possible to assess whether the policy has learned a genuine budget allocation strategy rather than only a shorter reasoning style.

5 Experimental Results

We evaluated our framework across varying model sizes, episode lengths, and hardware configurations. Our primary objective was to understand the relationship between model scale, compute constraints, and reward optimization. As our experiments scaled, the infrastructure evolved from small-scale validation pipelines to complex, distributed multi-GPU environments utilizing DeepSpeed and vLLM.

Experimental Configurations & Infrastructure

Our training runs utilized two primary hardware setups: $1 \times H100$ and $2 \times H100$ (80GB) configurations for smaller models (0.5B to 4B), and an $8 \times A100$ (80GB) cluster for larger 14B to 32B models. To accommodate the massive memory footprint of concurrent generation and policy updates for the Qwen3-14B model, we engineered an advanced distributed setup integrating DeepSpeed ZeRO-3 with CPU optimizer offloading. Furthermore, we isolated the workloads: GPUs 0–5 were dedicated to the PyTorch GRPO trainer, while GPUs 6–7 hosted a standalone vLLM inference server with a tensor parallel size of 2.

Initial Baselines and Hardware Bottlenecks As shown in Table 1, initial runs were conducted on a single H100 to validate the environment logic and training loop. Both the Qwen3-1.7B and Llama-3.2-1B-Instruct models successfully completed training pipelines but struggled to achieve positive average rewards. Further baseline experiments with Qwen2.5-3B-Instruct demonstrated noticeable learning progress, improving average re-

Model	Hardware	Infrastructure	Episode Length	Avg Reward	Outcome / Notes
Qwen3-1.7B	1×H100	Standard	Single	-0.560	Completed baseline
Llama-3.2-1B-Inst	1×H100	Standard	Single	-0.257	Completed baseline
Qwen2.5-3B-Inst	1×H100	Standard	Single	-0.148	Optimized baseline
Qwen3-4B	2×H100	Standard	10-Question	-8.595	Completed; 100% env success
Qwen2.5-14B-Inst	8×A100	FSDP Sharded	Single	0.260	Completed; first positive reward
Qwen3-14B	8×A100	ZeRO-3 + vLLM	4-Question	0.469	Completed 20/20 steps
Qwen3-14B	8×A100	ZeRO-3 + vLLM	10-Question	3.589	Partial (NCCL timeout)
Qwen2.5-32B	8×A100	ZeRO-3 + vLLM	–	–	Failed (NCCL / vLLM / context limitd)

Table 1: Comprehensive post-training results tracking model scale, infrastructure configuration, and episode length. Larger models combined with advanced sharding yielded the first positive reward signals, while long-horizon 10-question episodes generated the strongest potential rewards despite distributed system timeouts.

ward from -0.460 to -0.148 across runs, though it remained in the negative regime.

Because VRAM restrictions act as a severe bottleneck during concurrent solver and allocator steps, we scaled our approach to an 8×A100 configuration. Our first attempt at training the Qwen2.5-14B-Instruct model utilized standard FSDP model sharding. This increase in model capacity yielded a significant breakthrough: the model achieved a positive average reward of 0.260, indicating the emergence of basic budget-aware reasoning.

Advanced Episode Scaling and 14B Breakthrough Building on these initial baselines, we expanded our evaluation to test long-horizon stability using our advanced DeepSpeed/vLLM architecture. On the 2×H100 setup, Qwen3-4B successfully progressed to full 10-question rollouts (-8.595 reward), maintaining a 100% environment completion rate and proving the robustness of the core OpenEnv logic over extended sequences.

When applying the optimized 8×A100 ZeRO-3 + vLLM infrastructure to Qwen3-14B on true 4-question episodes with a 128-token cap, the model achieved our strongest completed large-model result. The run completed all planned optimizer steps over 480 logged episodes and 1,920 environment turns, reaching a positive mean episode reward of 0.469 with zero environment-step errors. This showed that the OpenEnv GRPO loop, vLLM server, and DeepSpeed ZeRO-3 trainer could operate together stably at 14B scale.

Key Findings and Distributed Instabilities Our consolidated results highlight a clear trend: scaling base model capacity dramatically improves the agent’s ability to optimize token budgets. When pushing the 14B model to "true 10-question" rollouts, the reward signal strengthened significantly, averaging 3.589 (with maximums reaching 10.78 and accuracy approaching 39%).

However, they should be interpreted as partial artifacts rather than completed training runs, because both attempts later failed from NCCL broadcast timeouts. Rollout logs showed no OpenEnv step errors or WebSocket reward failures, indicating that the bottleneck was distributed synchronization rather than the environment or reward function. Attempts to scale further to 32B met similar infrastructure limits, crashing due to vLLM KV-cache constraints and prompt-length bottlenecks. Stabilizing the distributed NCCL backend remains the primary necessary hurdle for future long-horizon training.

6 Conclusion and Future Work

We introduced **ReasoningEconomicEnv**, a reinforcement learning framework for training large language models to allocate reasoning tokens under a shared budget across multiple tasks. Unlike prior work that optimizes reasoning at the level of individual queries, our approach formulates reasoning as a sequential decision process, requiring the model to balance accuracy and compute usage over an entire episode. Through our environment design and GRPO-based training pipeline, we demonstrate that it is feasible to train models that exhibit non-trivial allocation behavior under realistic budget constraints.

Our experiments highlight both the promise and the infrastructure challenges of this setting. While smaller models (e.g., Qwen3-1.7B and 4B) successfully validated baseline environment stability across full 10-question rollouts, scaling to 14B parameters required overcoming severe memory bottlenecks. By engineering an advanced distributed pipeline integrating DeepSpeed ZeRO-3 with a decoupled vLLM generation server, we successfully trained Qwen2.5-14B-Instruct to convergence on multi-question episodes. This run achieved a stable,

positive average reward, demonstrating that larger models possess the capacity to internalize budget-aware meta-reasoning. Furthermore, exploratory 10-question rollouts at the 14B scale generated substantially stronger reward signals, confirming the value of long-horizon training constraints.

Despite these encouraging results, significant systems-level limitations remain. As observed in our extended 10-question 14B runs, the concurrent orchestration of long-context generation and policy optimization places immense strain on distributed backends, frequently resulting in NCCL synchronization timeouts. Furthermore, reward design remains an open challenge, as poorly calibrated penalties can lead to degenerate strategies such as under-utilizing compute or over-optimizing for efficiency.

Future work will focus on overcoming these distributed bottlenecks to unlock true long-horizon scaling. On the systems side, we plan to stabilize the PyTorch-vLLM communication layer to support 10+ question episodes for 14B and 32B models without NCCL timeouts. We will also explore parameter-efficient fine-tuning methods (e.g., LoRA/QLoRA) to further reduce VRAM pressure during these extended rollouts. On the modeling side, we aim to develop more robust reward functions and curriculum strategies that better guide the emergence of meta-reasoning behavior. Finally, we plan to extend evaluation beyond mathematical reasoning tasks to assess whether budget-aware allocation generalizes to broader domains, bringing us closer to the practical deployment of cost-aware LLM systems.

Individual Contributions

Harshawn Singh worked on post-training integration and debugging for ReasoningEconomicsPT/Env, including OpenEnv-backed GRPO rollout validation, multi-question episode mode, seed scouting, reward/ episode summaries, and rollout-debug tooling. He packaged the GPU workflow into a self-contained model-run bundle with Lambda/H100 and 8xA100 profiles, OpenEnv server wrappers, vLLM configuration, and DeepSpeed ZeRO-3 support. He validated local smoke tests, Qwen3-4B multi-question rollouts on H100s, and a completed Qwen3-14B run on 8xA100. He also tested longer true multi-question Qwen3-14B and Qwen2.5-32B soft-budget runs, which produced stronger partial reward signals, but ran into

some issues with NCCL timeouts and vLLM.

Khushi Kumari contributed to the post-training phase by running GRPO experiments with ReasoningEconomicsPT and ReasoningEconomicsEnv on Qwen2.5-3B-Instruct across both single- and multi-question episode configurations. Across progressive runs, she varied max_completion_length, max_tokens_per_step, and learning rate, observing a monotonic improvement in mean reward as context length and training duration increased, along with the emergence of positive-reward episodes. Her comparisons of hard-cap and soft-budget training modes showed that soft-budget settings reduced early episode termination and increased reward variance, making them a useful curriculum stage before applying strict budget constraints.

Prabhu Pugalenth acted as the primary project submitter and contributed to the post-training phase by running and debugging GRPO experiments using ReasoningEconomicsPT and ReasoningEconomicsEnv. Conducted smoke-test and training runs on a single H100 (80GB) with Qwen2.5-0.5B-Instruct, Qwen2.5-3B-Instruct, and Qwen2.5-7B-Instruct, and later tested model-sharded training on 8xA100 (80GB) GPUs with Qwen2.5-14B-Instruct, Qwen3-8B, and larger 30B/32B-scale models. Identified long generations, token-budget exhaustion, CUDA OOM, vLLM KV-cache allocation failures, and NCCL communicator issues as key bottlenecks for larger-model training, while also validating the environment locally and through the Hugging Face Space deployment and testing mitigations such as shorter-answer prompting, tokenizer-aligned budget computation, and multi-GPU model sharding.

Ryuichi Lun wrote the first drafts of the written and presentation materials, including the final presentation, midterm report, and final paper. Additionally, contributed to the post-training phase by running GRPO experiments using ReasoningEconomicsPT. Specifically, conducted training runs on a single H100 (80GB) using Qwen2-1.7B and Llama-3.2-1B-Instruct models, and performed model-sharded training on 8xA100 (80GB) GPUs with Qwen2.5-14B-Instruct. Identified VRAM limitations as a key bottleneck for larger models, and demonstrated that model sharding enabled stable training and improved average reward from negative to positive.

Yashaswi Sharma Built the V1 of the entire training pipeline. Built ReasoningEconomicsEnv, integrated the datasets, implemented the reward structure, and deployed the environment to [the corresponding Hugging Face space](#). Developed the ReasoningEconomicsPT post-training pipeline to unblock team in scaled-up deployments and make training easily configurable & extensible. Correspondingly, configured initial infrastructure deployments in single/multi-GPU settings for quick training iterations. Alongside the rest of the team, he shares ongoing responsibility for tracking down bugs, patching the environment logic, and pushing updates to the core infrastructure based on training feedback.

References

- Yichao Fu, Junda Chen, Siqi Zhu, Zheyu Fu, Zhongdongming Dai, Aurick Qiao, and Hao Zhang. 2024. Efficiently serving llm reasoning programs with certindex. *arXiv preprint arXiv:2412.20993*.
- Tingxu Han, Zhenting Wang, Chunrong Fang, Shiyu Zhao, Shiqing Ma, and Zhenyu Chen. 2025. Token-budget-aware llm reasoning. *Findings of the Association for Computational Linguistics: ACL 2025*.
- Sida Li, Jiazheng Wang, and Jingyang Yi. 2025. Self-budgeter: Adaptive token allocation for efficient llm reasoning. *arXiv preprint arXiv:2505.11274*.
- Haotian Luo, Li Shen, Shiwei Liu, Xiaochun Cao, and Dacheng Tao. 2025. O1-pruner: Length-harmonizing fine-tuning for o1-like reasoning. *arXiv preprint arXiv:2501.12570*.
- Xinyin Ma, Guangnian Wan, Runpeng Yu, Gongfan Fang, and Xinchao Wang. 2025. Cot-valve: Length-compressible chain-of-thought tuning. *arXiv preprint arXiv:2501.14249*.
- Niklas Muennighoff and 1 others. 2025. s1: Simple test-time scaling. *arXiv preprint arXiv:2501.19393*.
- Numina Project Team. 2024. Numinamath-tir dataset. <https://github.com/project-numina/numinamath>.
- Matthew Renze and Erhan Guven. 2024. The benefits of a concise chain of thought on problem-solving in large language models. *2024 2nd International Conference on Foundation and Large Language Models (FLLM)*.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*.
- Yi Shen, Jian Zhang, and 1 others. 2025. Dast: Difficulty-adaptive slow-thinking for large reasoning models. *arXiv preprint*.
- Charlie Snell, Jaehoon Lee, Hugo Touvron, and 1 others. 2024. Scaling test-time compute for reasoning in large language models. *arXiv preprint arXiv:2408.03314*.
- Xinyue Tong, Tianyi Zhang, and 1 others. 2024. Dart-math: Difficulty-aware reasoning training for mathematical problem solving. *arXiv preprint arXiv:2407.13690*.
- Heming Xia, Chak Tou Leong, Wenjie Wang, Yongqi Li, and Wenjie Li. 2025. Tokenskip: Controllable chain-of-thought compression in llms. *arXiv preprint*.
- Silei Xu, Wenhao Xie, Lingxiao Zhao, and Pengcheng He. 2025. Chain-of-draft: Thinking faster by writing less. *arXiv preprint arXiv:2502.18600*.
- Long Yu and 1 others. 2023. Metamath: Bootstrapping mathematical reasoning with meta-learning. *arXiv preprint arXiv:2309.12284*.